

CSC 591/791 & ECE 592: IoT Architectures, Applications, and
Implementation

IoT-Based Smart Wildfire Detection System

Website:

<https://iotproject-flask.onrender.com>

Team Members:

Sasidhar Appalla, Praveen Krishnan Meenakshi Sundar, Nikitha Maguluri, Randell Epps

Individual Contribution Breakdown

Table 1: Breakdown of individual contributions per project component

Component	Weight	Praveen	Sasidhar	Nikitha	Randell
Sensor Integration and Calibration	0.225	40%	30%	20%	10%
Real-Time Dashboard UI and Flask API	0.225	10%	40%	30%	20%
Dataset Curation and Risk Classifier Training	0.225	20%	10%	40%	30%
End-to-End UT	0.225	30%	20%	10%	40%
Documentation / Report Writing	0.10	25%	25%	25%	25%
Per-student total contribution		25%	25%	25%	25%

Abstract

Wildfires are an escalating threat worldwide, driven by climate change, drought, and human activity. To detect fires earlier and reduce response times, we designed a low-cost IoT system that fuses distributed edge sensing, real-time data ingestion, machine learning analytics, and automated notifications. Each ESP32-based node measures infrared radiation, humidity, and combustible gases and streams JSON-formatted readings to a Raspberry Pi, which writes them into a PostgreSQL database.

A Flask server loads a pre-trained Random Forest model (90 % balanced accuracy) to classify risk levels and dispatches WhatsApp alerts via Twilio whenever medium or high risk is detected. A web dashboard built with Bootstrap, Flask, Chart.js, and Leaflet visualizes live sensor data and geolocation, achieving an end-to-end latency of approximately 1.2 seconds. This architecture enables rapid situational awareness and scalable deployment for early wildfire response.

1 Introduction

Wildfires are becoming more frequent and intense due to climate change and prolonged droughts, making early detection—within minutes of ignition—essential for protecting lives and property. Traditional methods like satellite thermal imaging, lookout towers, and drone patrols suffer from high latency (satellites revisit every 8–12 hours), limited coverage, steep operational costs, and persistent false alarms. Ground-based cameras and towers require expensive infrastructure and staffing, while drones can’t guarantee continuous 24/7 monitoring and introduce logistical challenges.

To overcome these barriers, we designed a low-cost, distributed IoT network: ESP32 nodes sample infrared, humidity, and gas levels every few seconds and stream JSON payloads to a Raspberry Pi, which optionally geolocates the data and stores it in PostgreSQL. A Flask backend applies a pre-trained Random Forest model to predict risk.

1.1 Problem Statement

Traditional wildfire detection methods suffer from critical limitations:

- **High Latency:** Satellite thermal imaging only revisits each location every 8–12 hours, delaying alerts when every minute counts.
- **Limited Coverage:** Ground lookout towers and fixed cameras require expensive infrastruc-

ture and can’t monitor vast or obstructed terrain continuously.

- **Operational Cost:** Manned patrols, drone deployments, and high-resolution imagery services carry significant staffing and maintenance expenses.
- **Lack of Scalability:** Existing approaches cannot be deployed in large numbers across remote or rugged areas at low cost.

2 System Overview

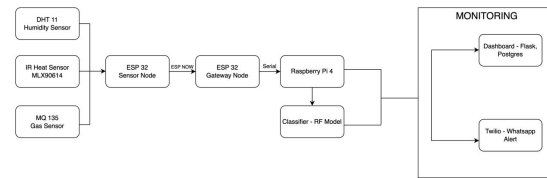


Figure 1: System Architecture Diagram

2.1 ESP32 Sensor Node

An ESP32 microcontroller interfaces with an IR sensor MLX90614, a DHT11 humidity sensor, and an MQ-135 gas sensor. Every five seconds it samples each sensor, timestamps the readings, and serializes them into a compact JSON payload. That payload is sent over serial.

2.2 Data Ingestion & Storage

A Raspberry Pi listens on the serial port, parses incoming JSON, and—if available—enriches it with Google Geolocation lookups. Validated records are inserted into PostgreSQL as JSONB, with B-tree and GIN indexes ensuring sub-millisecond retrieval of the latest or historical data.

2.3 Flask App

A Flask application exposes `/latest` and `/history?limit=N`. On each `/latest` call, it applies a cached Random Forest model (90 % balanced accuracy) to classify risk. Medium/High labels trigger Twilio WhatsApp alerts; a timestamp guard prevents duplicates. Deployed behind Gunicorn, the service handles concurrent dashboard polling easily.

2.4 Alerting & Visualization

Medium/High-risk events invoke Twilio’s WhatsApp API for geo-tagged alerts within seconds. Receipts and retries ensure reliability. A single-page dashboard (Bootstrap 5 + Chart.js + Leaflet) polls `/latest` every two seconds, updating summary cards, rolling charts, and a map. A history table of the last 100 readings provides context—all in a responsive UI.

3 Technologies Used

Python 3.9 A general-purpose scripting language that glues everything together—serial ingestion, database access, REST API, and alert logic—leveraging libraries like `psycopg2`, `requests`, and `paho-mqtt`.

ESP32 Microcontroller A low-cost, Wi-Fi-enabled SoC that reads IR, humidity (DHT11), and gas (MQ-135) sensors, packages readings as JSON, and supports OTA updates and deep-sleep power management for remote nodes.

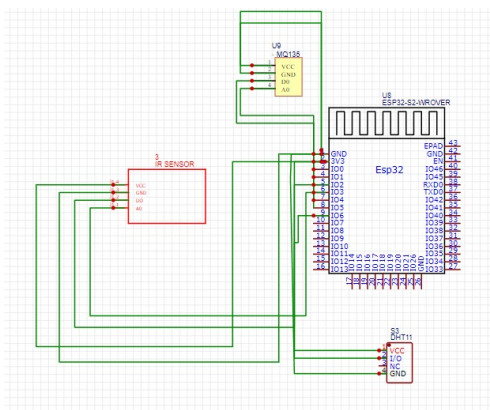


Figure 2: ESP32 node wiring: IR Heat Sensor, DHT11 humidity sensor, MQ-135 gas sensor, and Serial to Raspberry Pi.

Raspberry Pi (Model 3B/4B) The local data hub: listens on serial, optionally enriches with Google Geolocation, and writes JSON records into PostgreSQL via a threaded connection pool with robust error handling.

PostgreSQL (with JSON) Stores time-stamped sensor payloads and location data in a schemaless JSONB column, with B-tree and GIN indexes enabling sub-millisecond queries for both the latest reading and historical slices.

Flask Web Framework Serves the `/latest` and `/history` endpoints, loads the Random Forest model via `joblib`, classifies each reading, and triggers Twilio

alerts when needed; scales under Gunicorn for concurrent polling.

scikit-learn & joblib Provides the Random Forest classifier for wildfire-risk prediction and a label encoder; models are serialized with `joblib` for near-instant loading in the Flask app.

Bootstrap 5 A responsive CSS framework used in the dashboard’s HTML to arrange summary cards, grid layouts, and tables without hand-crafting complex styles.

Chart.js Renders animated, interactive line charts in the dashboard—plotting rolling windows of IR, humidity, and gas—complete with tooltips and smooth updates.

Leaflet.js Displays an interactive map (OpenStreetMap tiles) showing the sensor’s geolocation, with marker panning and popup tooltips for spatial context.

Twilio WhatsApp API Sends geo-tagged alerts to designated WhatsApp numbers; the Python client handles message templating, delivery callbacks, and retry logic to ensure reliability..

Google Geolocation API Converts Wi-Fi scan data into latitude/longitude coordinates, enhancing location accuracy without GPS hardware; can be disabled to avoid extra cost or privacy concerns.

Each of these technologies was chosen to balance cost, performance, and ease of development, resulting in a tightly integrated pipeline from sensor to alert.

3.1 Source-Code Files

File	Purpose
<code>espread5.py</code>	Reads serial JSON from ESP32, optional geolocation, inserts into PostgreSQL.
<code>app.py</code>	Flask API (<code>/latest</code> , <code>/history</code>), runs RF model, sends Twilio alerts.
<code>templates/index.html</code>	Dashboard UI (Bootstrap layout, Flask, Chart.js charts, Leaflet map, polling logic).
<code>rf_model.render.pkl</code>	Pre-trained Random Forest model for wildfire severity classification.
<code>label_encoder.render.pkl</code>	LabelEncoder mapping numeric classes to text labels.

Table 2: Source-code files and their roles

3.2 User-Controllable Parameters

Parameter	Default	Description
SERIAL_PORT	/dev/serial0	UART device for ESP32
BAUD_RATE	115200	Serial communication speed
PUBLISH_INTERVAL	5 s	Sensor sampling interval
GEO_API_URL	(keyed URL)	Enable/disable Google Geolocation lookups
POSTGRES_URL	env var	Database connection string
TWILIO_CREDS	env var	Twilio WhatsApp API SID, token, from/to
PORT	5000	Flask server port

Table 3: User-controllable configuration parameters

4 Results and Discussion

4.1 Predictive Performance

Table 4: Random Forest Classifier Metrics (test $n = 30$)

Label	Precision	Recall	F1-score
HighRisk	1.00	1.00	1.00
LowRisk	0.82	0.90	0.86
MediumRisk	0.88	0.78	0.82
Accuracy	0.90		

4.2 Dashboard Responsiveness

Concurrent polling by ten clients caused no perceptible lag; CPU usage remained below 25%.

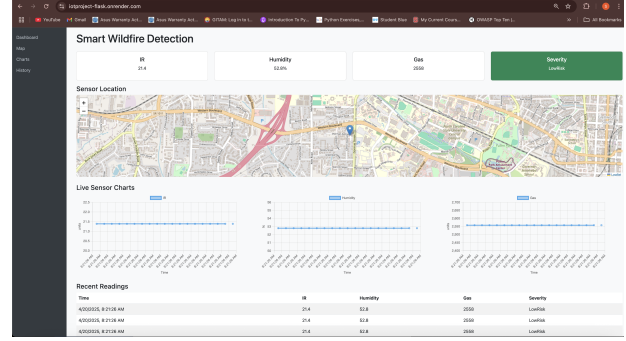


Figure 3: Web dashboard.

4.3 Limitations & Future Work

Real deployments may face RF dead-zones and environmental noise. Future work: mesh networking, adaptive thresholds, edge inference on ESP32.

5 Related Work and References

1. E. Barmounakis et al., "IoT-based wildfire detection: A review," *Sensors*, 21(4), 1175, 2021.
2. P. Sharma A. Kumar, "Real-time forest fire detection using ICARUS platform," *IEEE IoT J.*, 9(12), 8765–8774, 2022.
3. Y. Li et al., "Edge AI for forest fire monitoring," *ACM TOSN*, 19(3), 2023.
4. OpenStreetMap, "OpenStreetMap." <https://www.openstreetmap.org/>
5. Twilio, "WhatsApp API for Business." <https://www.twilio.com/whatsapp>
6. Flask Project, "Flask Documentation." <https://flask.palletsprojects.com/>
7. Chart.js, "Chart.js." <https://www.chartjs.org/>
8. Leaflet, "Leaflet." <https://leafletjs.com/>
9. Aosong Electronics, "AM2302/DHT11 Datasheet," 2015.
10. Winsen Electronics, "MQ-135 Gas Sensor Datasheet," 2014.